Description | Editorial | Solutions | Submissions

912. Sort an Array

Medium | Topics | Companies

Given an array of integers `nums`, sort the array in ascending order and return it.

You must solve the problem **without using any built-in** functions in `O(nlog(n))` time complexity and with the smallest space complexity possible.

Example 1:

```
Input: nums = [5,2,3,1]
Output: [1,2,3,5]
Explanation: After sorting the array, the positions of some numbers are not changed (for example, 2 and 3), while the positions of other numbers are changed (for example, 1 and 5).
```

Example 2:

```
Input: nums = [5,1,1,2,0,0]
Output: [0,0,1,1,2,5]
Explanation: Note that the values of nums are not necessarily unique.
```

Constraints:

- $1 <= nums.length <= 5 * 10^4$

Code

Java | Auto

```java
import java.util.*;
class Solution {
    public int[] sortArray(int[] nums) {
        Arrays.sort(nums);
        return nums;
    }
}
```

Saved
Ln 1, Col 20

Testcase | Test Result

Accepted Runtime: 1 ms

Case 1 | Case 2

Input

nums =
[5,2,3,1]

Output

[1,2,3,5]

Expected

[1,2,3,5]

Contribute a testcase

Description | Editorial | Solutions | Submissions

414. Third Maximum Number

Easy | Topics | Companies

Given an integer array `nums`, return *the **third distinct maximum** number in this array. If the third maximum does not exist, return the **maximum** number.*

Example 1:

```
Input: nums = [3,2,1]
Output: 1
Explanation:
The first distinct maximum is 3.
The second distinct maximum is 2.
The third distinct maximum is 1.
```

Example 2:

```
Input: nums = [1,2]
Output: 2
Explanation:
The first distinct maximum is 2.
The second distinct maximum is 1.
The third distinct maximum does not exist, so the maximum
(2) is returned instead.
```

Example 3:

```
Input: nums = [2,2,3,1]
Output: 1
Explanation:
```

3.5K 140 50 Online

Code

Java | Auto

```java
class Solution {
    public int thirdMax(int[] nums) {
        Integer max = null;
        Integer s_max = null;
        Integer t_max = null;
        for(Integer num : nums){
            if(num.equals(max) ||num.equals(s_max)||num.equals(t_max)){
                continue;
            }
            if(max==null || num>max){
                t_max = s_max;
                s_max = max;
                max =num;
            }
            else if(s_max==null || num>s_max){
                t_max = s_max;
                s_max = num;
            }
            else if(t_max==null || num>t_max){
                t_max = num;
            }
```

Saved

Ln 13, Col 26

Testcase | Test Result

Accepted Runtime: 0 ms

Case 1 | Case 2 | Case 3

Input

nums =

[3,2,1]

Problem List

Submit

Premium

Description | Accepted | Editorial | Solutions | Submissions

56. Merge Intervals

Solved

Medium | Topics | Companies

Given an array of `intervals` where `intervals[i] = [start_i, end_i]`, merge all overlapping intervals, and return *an array of the non-overlapping intervals that cover all the intervals in the input.*

Example 1:

```
Input: intervals = [[1,3],[2,6],[8,10],[15,18]]
Output: [[1,6],[8,10],[15,18]]
Explanation: Since intervals [1,3] and [2,6] overlap, merge them into [1,6].
```

Example 2:

```
Input: intervals = [[1,4],[4,5]]
Output: [[1,5]]
Explanation: Intervals [1,4] and [4,5] are considered overlapping.
```

Example 3:

```
Input: intervals = [[4,7],[1,4]]
Output: [[1,7]]
Explanation: Intervals [1,4] and [4,7] are considered overlapping.
```

24.4K | 266 | 409 Online

Code

Java | Auto

```java
import java.util.*;

class Solution {
    public int[][] merge(int[][] intervals) {
        Arrays.sort(intervals, (a, b) -> a[0] - b[0]);

        int n = intervals.length;
        int[][] res = new int[n][2];
        int idx = 0;

        res[0] = intervals[0];

        for (int i = 1; i < n; i++) {
            if (intervals[i][0] <= res[idx][1]) {
                // merge
                res[idx][1] = Math.max(res[idx][1], intervals[i][1]);
            } else {
                idx++;
                res[idx] = intervals[i];
            }
        }

```

Saved

Ln 26, Col 1

Testcase | Test Result

Accepted Runtime: 1 ms

Case 1 | Case 2 | Case 3

Input

intervals =

Description | Editorial | Solutions | Submissions

215. Kth Largest Element in an Array

Medium | Topics | Companies

Given an integer array `nums` and an integer `k`, return *the* k^{th} *largest element in the array.*

Note that it is the kth largest element in the sorted order, not the kth distinct element.

Can you solve it without sorting?

Example 1:

```
Input: nums = [3,2,1,5,6,4], k = 2
Output: 5
```

Example 2:

```
Input: nums = [3,2,3,1,2,4,5,5,6], k = 4
Output: 4
```

Constraints:

- $1 \le k \le nums.length \le 10^5$
- $-10^4 \le nums[i] \le 10^4$

18.6K 371 249 Online

Code

Java | Auto

```java
class Solution {
    public int findKthLargest(int[] nums, int k) {
            for (int i = 0; i < k; i++) {
            int maxIndex = i;
            for (int j = i + 1; j < nums.length; j++) {
                if (nums[j] > nums[maxIndex]) {
                    maxIndex = j;
                }
            }
            int temp = nums[i];
            nums[i] = nums[maxIndex];
            nums[maxIndex] = temp;
        }
        return nums[k - 1];
    }
}
```

Saved Ln 10, Col 32

Testcase | Test Result

Accepted Runtime: 0 ms

Case 1 | Case 2

Input

nums =
[3,2,1,5,6,4]

k =
2

Description | Editorial | Solutions | Submissions

1051. Height Checker

Easy | Topics | Companies | Hint

A school is trying to take an annual photo of all the students. The students are asked to stand in a single file line in **non-decreasing order** by height. Let this ordering be represented by the integer array `expected` where `expected[i]` is the expected height of the ith student in line.

You are given an integer array `heights` representing the **current order** that the students are standing in. Each `heights[i]` is the height of the ith student in line (**0-indexed**).

Return *the **number of indices** where* `heights[i] != expected[i]`.

Example 1:

```
Input: heights = [1,1,4,2,1,3]
Output: 3
Explanation:
heights:  [1,1,4,2,1,3]
expected: [1,1,1,2,3,4]
Indices 2, 4, and 5 do not match.
```

Example 2:

```
Input: heights = [5,1,2,3,4]
Output: 5
Explanation:
heights:  [5,1,2,3,4]
expected: [1,2,3,4,5]
```


Code

Java | Auto

```java
import java.util.*;

class Solution {
    public int heightChecker(int[] h) {
        int[] exp = h.clone();
        Arrays.sort(exp);
        int c = 0;
        for (int i = 0; i < h.length; i++) {
            if (h[i] != exp[i]) c++;
        }
        return c;
    }
}
```

Saved | Ln 14, Col 1

Testcase | Test Result

Accepted Runtime: 2 ms

Case 1 | Case 2 | Case 3

Input

heights =
[1,1,4,2,1,3]

Output

3

Expected